WORKSHOP M2 2026 · Workshop2026-M2

RESILIENCE-ALPHA

Dossier d'ingénierie et de gouvernance

« Tout système finit par tomber. La vraie question :
le vôtre saura-t-il se relever seul ? »

Plateforme de paiement fictive, conteneurisée et auto-résiliente

Piliers : DEV · INFRA · IA/DATA · CYBER · TRANSFORMATION

Équipe : CONTE · VERGNAULT · BULLION · LERUYET · DREVET · DIKEL

Octobre 2026

Sommaire

1. Synthèse . . . 3
2. Contexte et exigences . . . 3
3. Architecture . . . 5
4. Développement et infrastructure . . . 6
5. Détection par machine learning . . . 7
6. Sécurité . . . 8
7. Remédiation autonome . . . 9
8. Résultats mesurés . . . 10
9. Gouvernance : EBIOS RM, PCA/PRA, DORA/NIS2 . . . 11
10. Recommandations à la direction . . . 12
Annexe A — Décisions d'architecture . . . 13
Annexe B — Glossaire . . . 13

Comment lire ce dossier. Les sections 1 à 10 constituent le corps du dossier (8 à 12 pages). Les annexes (décisions d'architecture, glossaire) ne sont pas comptées. Toutes les valeurs mesurées proviennent de la campagne de chaos engineering du 19/10/2026 sur le poste de développement ; elles seront mises à jour avec les mesures de la recette sur le PC Hôte. Le code source fait foi : chaque affirmation renvoie à un fichier du dépôt.

1. Synthèse

Une plateforme de paiement subit des incidents à répétition, relancée chaque fois à la main. RESILIENCE-ALPHA prouve qu'un système critique peut **encaisser une attaque et se rétablir sans intervention humaine**. La plateforme détecte l'incident par apprentissage automatique, applique un plan de réponse déterministe, puis **prouve sa reprise par une nouvelle transaction réussie**.

1,8 s reprise après crash d'un service	**≈ 40 s** reprise après altération des données	**0** intervention humaine pendant la reprise	**19** indicateurs analysés par le ML toutes les 5 s

Ce que nous avons construit

- **Un SI de paiement codé par l'équipe** : front React, services d'authentification et de paiement en FastAPI, PostgreSQL. Aucune solution de paiement existante.
- **Une détection par machine learning** (Isolation Forest + enveloppe robuste apprises sur le trafic normal), et non par des seuils fixés à la main.
- **Un moteur de remédiation déterministe** : le ML qualifie l'incident, une politique YAML auditée choisit les actions. Le moteur n'a que des droits minimaux.
- **Une preuve de reprise automatique** : un incident n'est clos que lorsqu'une transaction témoin (connexion + virement + relecture) réussit à travers le WAF, après la dernière action.
- **Une gouvernance reliée au démonstrateur** : chaque scénario EBIOS RM correspond à un playbook ; les RTO du PCA/PRA sont mesurés par la plateforme ; la matrice DORA/NIS2 renvoie à des composants réels.

Ce qui reste à faire avant la soutenance

- Rejouer les scénarios Red Team depuis le **PC Attaquant physique** et mesurer leurs RTO.
- Vérifier l'IP source vue par le WAF depuis le hotspot (ADR-009).
- Tester l'assistant RAG de bout en bout avec Ollama sur le PC IA.

2. Contexte et exigences

Le sujet impose un SI de paiement fictif conteneurisé sur un PC Hôte, attaqué depuis un PC Attaquant distinct, avec détection par ML, remédiation automatique et preuve de reprise. Toute intervention humaine pendant la démo donne 0/5 sur l'axe remédiation. Le tableau ci-dessous relie chaque exigence à sa réalisation.

Exigence du sujet	Réalisation	Preuve dans le dépôt
SI de paiement conteneurisé (front, API, auth, BDD)	front, payment-service, auth-service, PostgreSQL	services/, infra/docker-compose.yml
Paiement codé par l'équipe	Virements idempotents, verrous ordonnés, anti-IDOR	services/payment-service/
Attaque réelle depuis un PC distinct	Kit Red Team portable (3 scénarios), copié sur le PC Attaquant, cible limitée au labo	attack/
Détection par ML, pas de seuils fixes	IsolationForest + enveloppe robuste, seuil auto-calibré	services/ml-detector/
Remédiation automatique	Politique YAML, 11 playbooks, garde-fous	services/remediation-engine/
Nouvelle transaction réussie après l'incident	Transaction témoin toutes les 5 s à travers le WAF	services/synthetic-probe/
Dashboard avant / pendant / après	Bandeau d'état, timeline, KPI temps réel	services/dashboard/

Exigence du sujet	Réalisation	Preuve dans le dépôt
RAG local Ollama + LangChain ≤ 8B	llama3.1:8b, réponses sourcées, aucun droit d'action	services/rag-assistant/

Les cinq piliers et leurs liens

Pilier	Socle technique	Fournit aux autres piliers
DEV	FastAPI, REST, React + Tailwind	Métriques et journaux métier, routes internes de remédiation
INFRA	Docker Compose, Ansible, Prometheus / Grafana / Loki	Les données de détection, le déploiement reproductible
IA / DATA	scikit-learn, LangChain + Ollama	Les événements qualifiés qui déclenchent la remédiation
CYBER	WAF ModSecurity, SIEM, durcissement, SOPS	Le point de blocage piloté par le moteur, le filet SIEM
TRANSFORMATION	EBIOS RM, PCA/PRA, DORA / NIS2	Les scénarios rejoués, les RTO cibles vérifiés par la sonde

3. Architecture

La plateforme tourne entièrement sur le PC Hôte (Docker Desktop + WSL2), soit une vingtaine de conteneurs. Le PC Attaquant n'accède qu'au port 443 du WAF, via le point d'accès Wi-Fi du labo. Le PC IA, facultatif, n'héberge que le modèle de langage.

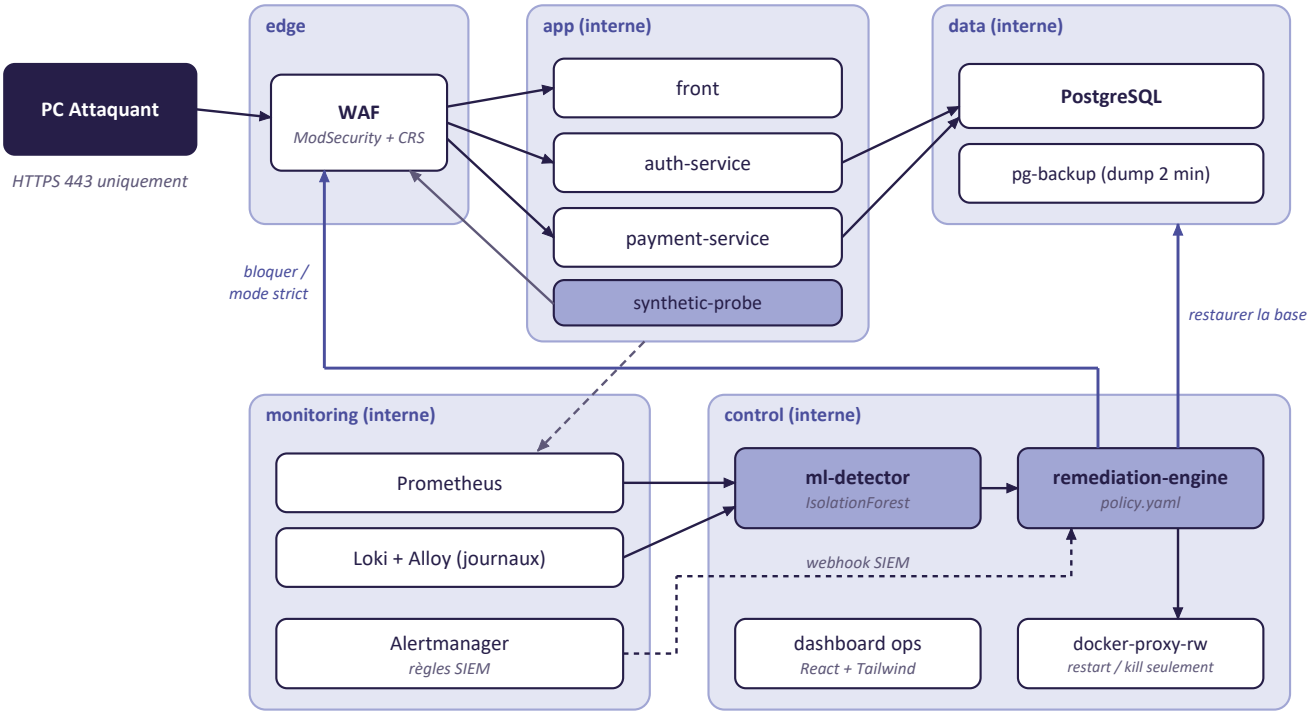


Figure 1 — Architecture logique : plan de données (haut) et plan de contrôle (bas). Flèches violettes : actions de remédiation ; pointillés gris : collecte des métriques et journaux.

Segmentation réseau

Réseau	Membres principaux	Rôle
edge	waf	Seul réseau relié à un port publié (443)
app (interne)	waf, front, auth, payment, sonde, traffic-gen	Trafic applicatif
data (interne)	auth, payment, postgres, pg-backup	Le WAF et le front n'atteignent jamais la base
monitoring (interne)	Prometheus, Loki, Alloy, exporters, ML	Collecte des métriques et journaux
control (interne)	moteur, ML, proxy Docker, dashboard, Alertmanager	Plan de contrôle séparé du plan de données
admin	Prometheus, Grafana, dashboard	Publiés sur 127.0.0.1 uniquement, sans NAT sortant

Principes : boucle fermée sans humain ; chemin de remédiation déterministe (le LLM n'y figure jamais) ; moindre privilège, en particulier pour le moteur ; tout est code (Compose, Ansible, règles, playbooks, dashboards).

4. Développement et infrastructure

Microservices (pilier DEV)

Service	Technologie	Fonctions clés
auth-service	FastAPI	Inscription, connexion, JWT RS256 + JWKS, argon2id, verrouillage progressif, révocation des sessions
payment-service	FastAPI + asyncpg	Comptes, virements idempotents (Idempotency-Key), verrous ordonnés, anti-IDOR, mode lecture seule, invariant comptable publié
front	React + Vite + Tailwind	Connexion, solde, virement, historique, rafraîchi en direct toutes les 3 s
dashboard	React + Tailwind	Bandeau d'état, KPI, courbes, timeline des incidents en temps réel (SSE)
synthetic-probe	Python	Connexion + virement + relecture toutes les 5 s, à travers le WAF
traffic-gen	Python	20 utilisateurs simulés : trafic de référence pour l'apprentissage du ML
pg-backup	Python + pg_dump	Dump toutes les 2 min, marqué « sain » seulement si la sonde est OK ; restauration à la demande

Les services communiquent en REST/JSON (ADR-002) : le WAF inspecte nativement ce format, et les journaux restent lisibles. Toutes les requêtes SQL sont paramétrées.

Infrastructure (pilier INFRA)

- **Docker Compose** (*infra/docker-compose.yml*) : un seul fichier décrit la plateforme, avec healthchecks, limites de ressources, redémarrage automatique et ordre de démarrage explicite ; les images maison sont toujours construites sur place (*pull_policy: build*).
- **Ansible** : *deploy.yml* (déploiement avec fenêtre de maintenance pour éviter que le moteur ne « répare » un redéploiement volontaire), *reset_demo.yml* (remise à zéro avant la démo), *teardown.yml*.
- **Observabilité** : Prometheus (métriques), Loki + Alloy (journaux de tous les conteneurs), Grafana (dashboard provisionné), Alertmanager (routage des alertes SIEM), postgres-exporter.
- **Sauvegardes** : rétention 1 h ; seule une sauvegarde prise pendant que la sonde réussit peut servir à restaurer.

5. Détection par machine learning

Un seuil fixe est contourné par un attaquant qui reste juste en dessous, et déclenche à tort lors d'un pic légitime. Nous apprenons plutôt le comportement normal de la plateforme, puis mesurons l'écart.

Apprentissage automatique de la référence

Au premier démarrage, le *ml-detector* collecte 120 fenêtres saines de 5 s (≈ 10 min) produites par *traffic-gen*, en écartant celles où la sonde échoue. Il entraîne deux modèles complémentaires et calibre le seuil au-dessus du pire score observé (+3 %), pour viser zéro faux positif. Le modèle est conservé dans un volume : un redémarrage ne nécessite pas de réapprentissage.

Modèle	Ce qu'il détecte	Pourquoi
IsolationForest + RobustScaler	Combinaisons inhabituelles de trafic, latence, ressources	Non supervisé : voit aussi l'inconnu ; léger
Enveloppe robuste	Signaux constants en temps normal : sonde en échec, service ou base arrêtés, invariant comptable	L'Isolation Forest ne sait pas découper une feature constante (bug trouvé en test)

19 indicateurs, calculés toutes les 5 s sur une fenêtre de 30 s

Famille	Indicateurs
Trafic	débit WAF, ratios 4xx / 5xx, taux de 403, clients distincts, part du client le plus actif, entropie des chemins
Authentification	ratio d'échecs de connexion, débit de tentatives
Performance et ressources	latence p95 de l'API, CPU et mémoire des conteneurs métier, connexions PostgreSQL
Métier et données	débit de paiements, modifications de lignes par paiement, **écart à l'invariant comptable**
Santé	échec de la sonde, cibles arrêtées, base injoignable

Qualification de l'incident

Le type d'incident choisit le playbook. Les signatures spécifiques sont testées en priorité : base arrêtée → *db_outage* ; invariant comptable violé → *data_tampering* ; service arrêté → *service_crash* ; pic d'échecs de connexion → *brute_force* ; pic de blocages 403 → *injection* ; sinon attribution au groupe d'indicateurs le plus écarté (*http_flood*, *resource_exhaustion*). Un classifieur supervisé RandomForest (*train.py*) remplace cette attribution dès que les campagnes étiquetées (*attack/data/labels.csv* et *chaos/*, fusionnés dans *ml/data/labels.csv* sur le PC Hôte) sont disponibles. Un mécanisme d'**épisode d'anomalie** émet un seul événement par incident, pour éviter les doublons.

Assistant RAG local

LangChain + Ollama, modèle *llama3.1:8b* (≤ 8 milliards de paramètres). Il indexe la documentation, la politique de remédiation et les incidents, répond en citant ses sources et rédige un **brouillon** de notification d'incident au format DORA/NIS2. Il n'a **aucun droit d'action** (ADR-008) : une injection de prompt via les journaux ne peut rien déclencher. Statut : code prêt, test de bout en bout à faire sur le PC IA.

6. Sécurité

Modèle de menace

Actif	Menace	Prévention	Détection	Playbook
Comptes clients	Bourrage d'identifiants	Rate-limit login, argon2id, verrouillage	ML + SIEM AuthFailureSurge	brute_force
API de paiement	Injection	WAF CRS, liste blanche des routes, SQL paramétré	ML + SIEM WafBlockSurge	injection
Disponibilité	Saturation HTTP	Rate-limit, limites CPU / mémoire	ML (débit, latence)	http_flood
Soldes	Altération des données	Rôles BDD à moindre privilège	ML (invariant) + SIEM	data_tampering
Composants	Panne, crash	Redémarrage auto, healthchecks	Docker + SIEM	service_crash, db_outage

Scénarios Red Team

Trois scénarios, rejoués depuis le PC Attaquant : bourrage d'identifiants (SO1), tentatives d'injection (SO2) et saturation HTTP (SO3). Le kit *attack/* est **portable** : il ne dépend que de la bibliothèque standard de Python et se copie seul sur le PC Attaquant. Il vise par défaut le PC Hôte (192.168.137.1, port 443) et refuse toute autre cible. Il fonctionne en **mode aveugle** : le dashboard n'étant joignable que depuis le PC Hôte, l'attaquant lance l'attaque sans voir la défense, et le temps de reprise se lit sur l'écran défenseur.

WAF

Nginx + ModSecurity v3 + OWASP CRS 4 (paranoïa 1), seul point d'entrée exposé. TLS 1.2/1.3, en-têtes de sécurité (HSTS, CSP stricte, X-Frame-Options), liste blanche explicite des routes de l'API : tout autre chemin */api/** (dont les routes internes) renvoie 404. Rate-limit à 5 req/s sur la connexion et 30 req/s sur l'API. Le moteur pilote trois contrôles dynamiques, rechargés à chaud par SIGHUP : liste de blocage IP, **empreinte IP + User-Agent** et mode strict à 3 req/s. La sonde et le trafic interne en sont exemptés.

Derrière Docker Desktop, le WAF voit la passerelle 172.30.0.1 au lieu de l'IP du client. Bloquer cette IP couperait tous les clients : le moteur bloque alors l'empreinte (ADR-009).

SIEM

SIEM léger construit sur la pile d'observabilité (ADR-007) : 4 règles Prometheus (sonde en échec, service ou base arrêtés, sauvegardes en échec) et 3 règles Loki (pic d'échecs de connexion, pic de blocages WAF, commandes SQL destructives), routées par Alertmanager vers le moteur. Le ML reste la source principale ; le SIEM est un **filet de sécurité** qui fonctionne même si le ml-detector tombe.

Durcissement et secrets

- Tous les conteneurs : *read_only*, *cap_drop: ALL*, *no-new-privileges*, limites mémoire ; services applicatifs en utilisateur non-root.
- Sept réseaux segmentés, dont quatre internes sans accès Internet ; seul le 443 est exposé.
- Un rôle PostgreSQL par service, au moindre privilège ; journal des écritures sans valeur de paramètre.
- Secrets générés localement (*scripts/init_secrets.py*), injectés en Docker secrets (jamais en variable d'environnement), ignorés par git ; partage dans l'équipe chiffré avec **SOPS + age**. Vault est l'évolution prévue (ADR-006).

7. Remédiation autonome

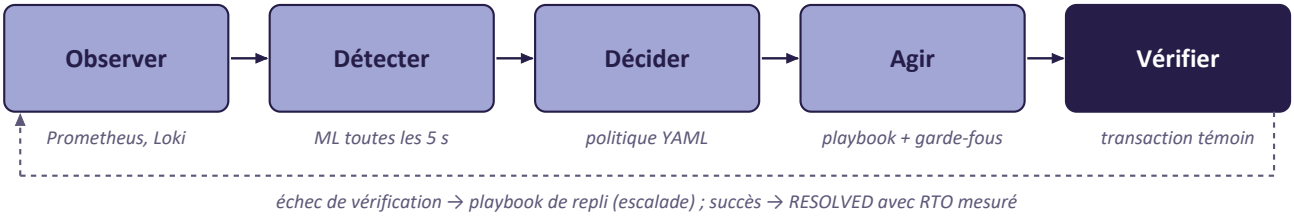


Figure 2 — La boucle fermée : un incident n'est résolu que lorsque la vérification réussit.

Trois sources alimentent le moteur : le **ML** (source principale), la **surveillance Docker** et le **SIEM**. Les événements du même type sont regroupés dans un seul incident (fenêtre de 120 s). Le moteur applique le playbook prévu par *policy.yaml*, puis attend une transaction témoin réussie, démarrée après la dernière action. En cas d'échec, il enchaîne un playbook de repli (2 au maximum), puis passe l'incident en *ESCALATED*.

Type d'incident	Actions automatiques	Repli
brute_force	Bloquer la source (15 min), révoquer les sessions concernées	flood_protection
injection	Bloquer la source (30 min)	data_tampering
http_flood	Bloquer la source (10 min), mode strict (5 min)	service_crash
resource_exhaustion	Redémarrer le conteneur suspect, mode strict	restart_business
service_crash	Redémarrer le conteneur suspect	restart_business
db_outage	Redémarrer PostgreSQL	restore_db
data_tampering	Lecture seule → bloquer la source → restaurer le dernier dump sain → révoquer toutes les sessions → réouverture	—

Accès du moteur maîtrisés

Le moteur peut redémarrer des conteneurs et restaurer la base : c'est le composant le plus dangereux. Ses accès sont donc limités au strict nécessaire.

Mesure	Détail
Pas de socket Docker	Proxy à expressions régulières : seuls GET /containers/json et POST /containers/<id>/(restart|kill) sont autorisés, et seulement depuis le moteur. Ni create, ni exec, ni delete
Cibles en liste blanche	Seuls les conteneurs portant le label resilience.remediable=true peuvent être visés
Liste de protection	Réseaux internes, sonde, trafic de référence et conteneurs critiques ne sont jamais ciblés
Actions figées	Seules les actions codées dans actions.py existent ; la politique est intégrée à l'image
Budget anti-boucle	15 actions par 5 min au maximum, 2 escalades par incident ; au-delà, ESCALATED
Secrets minimaux	Le moteur ne détient qu'un jeton de service interne : ni mot de passe de base, ni clé JWT
Traçabilité	Chaque action est journalisée (source, cible, résultat) et persistée (SQLite, reprise après crash)

8. Résultats mesurés

Campagne de chaos engineering (*chaos/chaos.py*) du 19/10/2026. Le RTO est mesuré automatiquement du début de la fenêtre anormale jusqu'à la transaction témoin réussie.

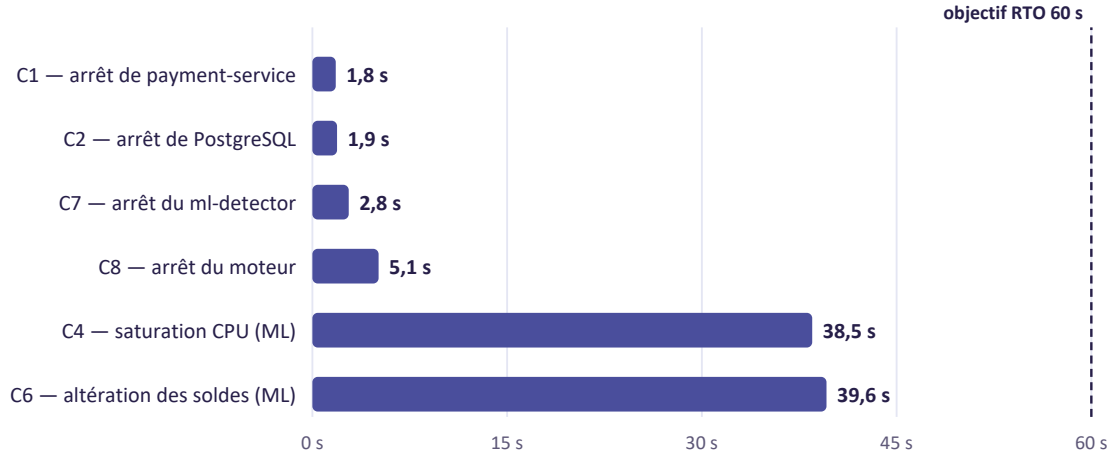


Figure 3 — Temps de reprise (RTO) mesuré par scénario. Les six restent sous l'objectif de 60 s du PCA.

Scénario	Détecté par	Type qualifié	Actions automatiques
C1 — arrêt de payment-service	Docker	service_crash	Redémarrage
C2 — arrêt de PostgreSQL	Docker	db_outage	Redémarrage de la base
C4 — saturation CPU d'auth-service	ML (cpu_business)	resource_exhaustion	Redémarrage + mode strict
C6 — mise à zéro de tous les soldes	ML (score 1,0, ledger_drift)	data_tampering	Lecture seule, restauration (0,14 s), révocation globale, réouverture
C7 / C8 — arrêt du ML / du moteur	—	—	Relancés par Docker, état et modèle repris

Autres vérifications : aucun incident ML en régime normal (score ≈ 0,37-0,50 pour un seuil de 0,6) ; un seul incident par injection ; blocage par empreinte ciblé (les autres clients derrière la même IP passent) ; aucun faux incident pendant un redéploiement sous fenêtre de maintenance.

Pourquoi ≈ 40 s pour les scénarios ML ? Fenêtre glissante de 30 s + 2 fenêtres consécutives (anti-rebond contre les faux positifs) + actions + transaction témoin. Piste : fenêtre de 15 s, ou une seule fenêtre pour une violation de l'invariant comptable.

Bugs trouvés par les tests et corrigés

- Altération des soldes non détectée → ajout de l'enveloppe robuste et de l'invariant comptable.
- Une injection produisait 2 ou 3 incidents → notion d'épisode d'anomalie côté ML, rattachement côté moteur.
- Le moteur « réparait » les redéploiements → fenêtre de maintenance pilotée par Ansible.
- Moteur « zombie » après arrêt → délai d'arrêt borné et chien de garde interne.

9. Gouvernance : EBIOS RM, PCA/PRA, DORA/NIS2

Analyse de risques EBIOS RM

Valeurs métier : exécuter un paiement, authentifier les clients, conserver l'historique, superviser et rétablir. Sources de risque retenues : cybercriminel (gain financier), hacktiviste (atteinte à l'image), initié malveillant (justifie le moindre privilège du moteur), défaillance non intentionnelle. Événements redoutés : indisponibilité > 5 min (grave), altération des soldes (critique), usurpation de comptes (grave), fuite de données (grave), perte de la capacité de reprise (grave).

Scénario opérationnel	Événement redouté	Détection	Playbook	Vraisemblance
SO1 — prise de contrôle de comptes	Usurpation	ML brute_force	brute_force	V3 → V1
SO2 — exploitation de l'API	Altération, fuite	ML injection + WAF	injection	V2 → V1
SO3 — saturation du service	Indisponibilité	ML http_flood	http_flood	V3 → V2
SO4 — altération des données	Altération	ML data_tampering	data_tampering	V2 → V1
SO5 — panne d'un composant	Indisponibilité, perte de reprise	Docker + ML	service_crash, db_outage	V3 → V1

Risque résiduel assumé : le PC Hôte unique est un point unique de défaillance physique, traité dans les recommandations.

PCA / PRA

Service	RTO cible	RPO	Mécanisme	Mesuré
Paiement	60 s	0	Remédiation automatique + redémarrage	1,8 s (crash)
Authentification	60 s	0	Redémarrage, révocation des sessions	38,5 s (saturation)
Données	2 min	≤ 2 min	Dump toutes les 2 min + restauration du dernier dump sain	39,6 s (altération)
Supervision	30 s	—	Redémarrage automatique, état persistant	2,8-5,1 s

Continuité : mode lecture seule pendant une restauration (le client consulte son solde), mode strict qui préserve les clients légitimes pendant un flood. **Reprise** : isoler la source, restaurer, révoquer, vérifier par transaction témoin, rouvrir. Les tests de chaos **sont** nos exercices de PRA.

Conformité DORA / NIS2

Une plateforme de paiement relève de **DORA** (règlement UE 2022/2554, applicable depuis le 17 janvier 2025), qui fait office de *lex specialis* par rapport à **NIS2** (directive UE 2022/2555) pour la gestion des risques TIC et la notification des incidents.

Exigence	Texte	Réponse du démonstrateur
Cadre de gestion du risque TIC	DORA art. 5-6	Analyse EBIOS RM, politique de remédiation versionnée
Protection et prévention	DORA art. 9 · NIS2 art. 21	WAF, segmentation, durcissement, moindre privilège, SOPS
Détection des anomalies	DORA art. 10	ML + SIEM, supervision continue
Réponse et rétablissement	DORA art. 11-12	Remédiation automatique, sauvegardes, restauration testée, RTO mesurés
Gestion des incidents	DORA art. 17-18	États d'incident, journal d'audit horodaté
Notification	DORA art. 19 · NIS2 art. 23	Brouillon de rapport généré par le RAG, relu par un humain
Tests de résilience	DORA art. 24-25	Chaos engineering, rejeu des scénarios EBIOS
Risque lié aux tiers	DORA art. 28	Images versionnées ; LLM local, aucune donnée envoyée à un tiers

10. Recommandations à la direction

#	Constat	Recommandation	Gain en risque
1	Les reprises manuelles sont lentes et non reproductibles	Généraliser l'auto-remédiation aux services critiques, avec une politique revue en comité des risques	RTO de dizaines de minutes à moins d'une minute
2	Un hôte unique est un point unique de défaillance	Deux hôtes au minimum, base répliquée, PRA multi-site	Supprime ER1 en cas de panne matérielle
3	La résilience non testée se dégrade	Chaos engineering mensuel, conforme aux tests exigés par DORA	Détecte les régressions avant les incidents réels
4	Le ML et le LLM sont de nouveaux risques	Modèles versionnés, carte de modèle, revue des faux positifs ; LLM hors du chemin de décision	Explicabilité face au régulateur
5	Les délais de notification sont courts	Rapports DORA/NIS2 pré-remplis par l'outillage, astreinte pour les incidents ESCALATED	Respect des délais réglementaires

Limites et prochaines étapes

- Intégrer Vault (secrets dynamiques, rotation des secrets comme action de remédiation).
- Épingler les images par digest et les scanner avec Trivy ; ajouter un hook gitleaks.
- Injecter de la latence réseau (scénario C3) et entraîner le classifieur supervisé sur les campagnes réelles.
- Réduire le RTO des scénarios ML (fenêtre de 15 s).

Contributions individuelles

Membre	Pilier / rôle	Contributions principales
DREVET	Chef de projet / architecte	Architecture globale, intégration de bout en bout, décisions d'architecture (ADR), moteur de remédiation
CONTE	DEV	auth-service, payment-service, front client et dashboard ops (React + Tailwind), transaction témoin
BULLION	INFRA	Docker Compose, playbooks Ansible, observabilité Prometheus / Loki / Grafana, sauvegardes pg-backup
VERGNAULT	IA / DATA	ml-detector (indicateurs, Isolation Forest, qualification), classifieur supervisé, assistant RAG
LERUYET	CYBER	WAF ModSecurity, règles SIEM, durcissement des conteneurs, gestion des secrets, scénarios Red Team
DIKEL	TRANSFORMATION	Analyse EBIOS RM, PCA/PRA, matrice DORA / NIS2, recommandations à la direction

Annexe A — Décisions d'architecture

ADR	Décision	Alternative écartée
001	Python/FastAPI pour tous les services backend	Node.js, Go
002	REST/JSON plutôt que gRPC	gRPC : moins lisible pour le WAF et les journaux
003	Détection en deux étages (non supervisé + supervisé)	Seuils fixes (interdits), LSTM (trop long)
004	Politique de remédiation déclarative et déterministe	Décision par LLM (non déterministe, injection de prompt)
005	Proxy Docker à liste blanche par regex	Socket monté (= root sur l'hôte), tecnativa (autorise containers/create)
006	Docker secrets + SOPS ; Vault en évolution	Vault en mode dev (non sécurisé)
007	SIEM léger Loki + Alertmanager	Wazuh, Elastic (≈ 4 Go de RAM en plus)
008	LLM hors du chemin critique	—
009	Blocage par empreinte IP + User-Agent derrière le NAT	Blocage IP seul (couperait tous les clients)
010	Grafana Alloy plutôt que Promtail	Promtail (fin de vie)
011	Référence ML apprise automatiquement sur périodes saines	Jeu de données préalable
012	Reprise vérifiée par transaction témoin réelle	Simple healthcheck

Annexe B — Glossaire

Terme	Définition
RTO / RPO	Durée maximale d'interruption / perte de données maximale acceptables
PCA / PRA	Plan de continuité / de reprise d'activité
WAF	Pare-feu applicatif web, filtre les requêtes HTTP malveillantes
SIEM	Corrélation des journaux et métriques pour détecter les incidents de sécurité
EBIOS RM	Méthode d'analyse de risques de l'ANSSI
DORA / NIS2	Règlement européen sur la résilience opérationnelle numérique du secteur financier / directive sur la cybersécurité
Isolation Forest	Algorithme non supervisé qui isole les observations atypiques
RAG	Génération augmentée par recherche : un LLM répond à partir de documents retrouvés
Transaction témoin	Virement automatique de test qui prouve que le service fonctionne réellement